

Assignment 1

Implement Inheritance Types Using JavaScript

Subject: Full Stack Web Development

Roll No: **A24126552267**

1. TITLE

Implement Inheritance Types Using JavaScript

2. AIM

To implement and demonstrate different types of inheritance in JavaScript using simple ES6 class examples and mixin techniques.

3. OBJECTIVES

- To understand inheritance and code reusability in JavaScript.
- To implement single inheritance using the extends keyword.
- To implement multilevel and hierarchical inheritance.
- To understand multiple inheritance behavior using mixins.
- To combine inheritance patterns to implement hybrid inheritance.
- To execute JavaScript programs using Node.js and observe the results.

4. DESCRIPTION / THEORY

Inheritance allows one class to reuse the properties and methods of another class. The existing class is called the parent or base class, while the class that inherits from it is called the child or derived class. In JavaScript ES6, inheritance is commonly created using the **class** syntax and the **extends** keyword.

The assignment demonstrates five inheritance patterns:

Single Inheritance: A child class inherits features from one parent class. Here, SavingsAccount inherits the displayBalance() method from Account and adds its own addInterest() method.

Multilevel Inheritance: A class inherits from another derived class, forming a chain. Here, Device is inherited by SmartDevice, and SmartDevice is inherited by SmartWatch.

Hierarchical Inheritance: Multiple child classes inherit from one common parent class. Here, EmailNotification and SMSNotification both inherit the send() method from Notification.

Multiple Inheritance (Using Mixins): JavaScript classes cannot directly extend multiple classes. Mixins can combine reusable behaviors from multiple objects. Here, a Robot gets both scanning and reporting abilities.

Hybrid Inheritance: Hybrid inheritance combines more than one inheritance pattern. This example uses a class hierarchy and adds an extra reusable mixin behavior to the child class.

5. KEYWORDS / IMPORTANT CONCEPTS USED

Keyword / Concept	Meaning
Class	A blueprint used to create objects.
Object	An instance created from a class.
Inheritance	Reusing properties and methods of another class.
Parent Class	The class from which another class inherits.
Child Class	The class that inherits features from a parent class.
extends	Keyword used to create class inheritance in JavaScript.

Keyword / Concept	Meaning
Method	A function defined inside a class or object.
Mixin	A reusable object whose methods can be added to another class.
Object.assign()	Used to copy mixin methods into a class prototype.
console.log()	Displays program output in the terminal.
Node.js	Used to run JavaScript programs outside the browser.

6. PROGRAMS

The following programs are simple examples created specifically for this assignment. They demonstrate the inheritance concepts using practical objects instead of the usual routine Animal–Dog examples.

1. Single Inheritance

Description: A child class inherits features from one parent class. Here, SavingsAccount inherits the displayBalance() method from Account and adds its own addInterest() method.

File Name: 01_single_inheritance.js

JavaScript Code:

```
class Account {
  constructor(balance) {
    this.balance = balance;
  }

  displayBalance() {
    console.log("Balance:", this.balance);
  }
}

class SavingsAccount extends Account {
  addInterest() {
    this.balance += 100;
    console.log("Interest added");
  }
}

const user = new SavingsAccount(1000);
user.displayBalance();
user.addInterest();
user.displayBalance();
```

2. Multilevel Inheritance

Description: A class inherits from another derived class, forming a chain. Here, Device is inherited by SmartDevice, and SmartDevice is inherited by SmartWatch.

File Name: 02_multilevel_inheritance.js

JavaScript Code:

```
class Device {  
  powerOn() {  
    console.log("Device is ON");  
  }  
}  
  
class SmartDevice extends Device {  
  connectWiFi() {  
    console.log("WiFi connected");  
  }  
}  
  
class SmartWatch extends SmartDevice {  
  showTime() {  
    console.log("Time displayed");  
  }  
}  
  
const watch = new SmartWatch();  
watch.powerOn();  
watch.connectWiFi();  
watch.showTime();
```

3. Hierarchical Inheritance

Description: Multiple child classes inherit from one common parent class. Here, EmailNotification and SMSNotification both inherit the send() method from Notification.

File Name: 03_hierarchical_inheritance.js

JavaScript Code:

```
class Notification {
  send() {
    console.log("Notification sent");
  }
}

class EmailNotification extends Notification {
  email() {
    console.log("Email notification");
  }
}

class SMSNotification extends Notification {
  sms() {
    console.log("SMS notification");
  }
}

const email = new EmailNotification();
email.send();
email.email();

const sms = new SMSNotification();
sms.send();
sms.sms();
```

4. Multiple Inheritance (Using Mixins)

Description: JavaScript classes cannot directly extend multiple classes. Mixins can combine reusable behaviors from multiple objects. Here, a Robot gets both scanning and reporting abilities.

File Name: 04_multiple_inheritance.js

JavaScript Code:

```
const Scanner = {  
  scan() {  
    console.log("Scanning object");  
  }  
};  
  
const Reporter = {  
  report() {  
    console.log("Report generated");  
  }  
};  
  
class Robot {}  
  
Object.assign(Robot.prototype, Scanner, Reporter);  
  
const robot = new Robot();  
robot.scan();  
robot.report();
```

5. Hybrid Inheritance

Description: Hybrid inheritance combines more than one inheritance pattern. This example uses a class hierarchy and adds an extra reusable mixin behavior to the child class.

File Name: 05_hybrid_inheritance.js

JavaScript Code:

```
class User {
  login() {
    console.log("User logged in");
  }
}

class Student extends User {
  study() {
    console.log("Student is studying");
  }
}

const CanDownload = {
  download() {
    console.log("Material downloaded");
  }
};

class OnlineStudent extends Student {}
Object.assign(OnlineStudent.prototype, CanDownload);

const learner = new OnlineStudent();
learner.login();
learner.study();
learner.download();
```


7. CONCLUSION

The programs demonstrate how inheritance can be used to reuse existing code and extend functionality. JavaScript supports common class inheritance with `extends`, while mixins provide a simple way to combine multiple reusable behaviors. These concepts help create clean, modular, and maintainable applications.